data structure and reduce the time an individual thread spends waiting, it may well *decrease* overall performance. First, the atomic operations used for lock-free code can be much slower than nonatomic operations, and there'll likely be more of them in a lock-free data structure than in the mutex locking code for a lock-based data structure. Not only that, but the hardware must synchronize data between threads that access the same atomic variables. As you'll see in chapter 8, the cache ping-pong associated with multiple threads accessing the same atomic variables can be a significant performance drain. As with everything, it's important to check the relevant performance aspects (whether that's worst-case wait time, average wait time, overall execution time, or something else) both with a lock-based data structure and a lock-free one before committing either way.

Now let's look at some examples.

7.2. Examples of lock-free data structures

In order to demonstrate some of the techniques used in designing lock-free data structures, we'll look at the lock-free implementation of a series of simple data structures. Not only will each example describe the implementation of a useful data structure, but I'll use the examples to highlight particular aspects of lock-free data structure design.

As already mentioned, lock-free data structures rely on the use of atomic operations and the associated memory-ordering guarantees in order to ensure that data becomes visible to other threads in the correct order. Initially, we'll use the default `memory_order_seq_cst` memory ordering for all atomic operations, because that's the easiest to reason about (remember that all `memory_order_seq_cst` operations form a total order). But for later examples we'll look at reducing some of the ordering constraints to `memory_order_acquire`, `memory_order_release`, or even `memory_order_relaxed`. Although none of these examples use mutex locks directly, it's worth bearing in mind that only `std::atomic_flag` is guaranteed not to use locks in the implementation. On some platforms what appears to be lock-free code might actually be using locks internal to the C++ Standard